

# Guardian — plan de código

CALL ME MAYBE · BLOQUE 4 · 2026-08-27

Una sección por método, en el orden en que conviene escribirlos. Cada una trae la **firma**, lo que **recibe**, lo que **devuelve** y los **pasos**, ya decididos. Aquí no queda nada que pensar: se teclea.

Attributes

EL ESTADO COMPLETO

`_vocab · _reversed_vocab · _functions · _json_str · _stack · _slot · _written · _done`

| ATRIBUTO                     | TIPO                                               | EN REPOSO                                 | QUÉ GUARDA                                                                                                  |
|------------------------------|----------------------------------------------------|-------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <code>_vocab</code>          | <code>Dict[str, int]</code>                        | <code>del</code><br><code>__init__</code> | texto → id                                                                                                  |
| <code>_reversed_vocab</code> | <code>Dict[int, str]</code>                        | <code>del</code><br><code>__init__</code> | id → texto                                                                                                  |
| <code>_functions</code>      | <code>Dict[str, Function]</code>                   | <code>del</code><br><code>__init__</code> | nombre → schema                                                                                             |
| <code>_json_str</code>       | <code>str</code>                                   | <code>""</code>                           | El JSON que se va escribiendo                                                                               |
| <code>_stack</code>          | <code>List[Tuple[Dict[str, TypeSpec], int]]</code> | <code>[]</code>                           | Un nivel abierto por entrada: el <b>nodo</b> del que salen las claves + el <b>índice</b> de la clave actual |
| <code>_slot</code>           | <code>Optional[str]</code>                         | <code>None</code>                         | <code>None · "name" · "number" · "string"</code>                                                            |
| <code>_written</code>        | <code>str</code>                                   | <code>""</code>                           | Lo que el modelo lleva escrito <b>dentro del hueco actual</b> . Se vacía al abrir cada hueco                |
| <code>_done</code>           | <code>bool</code>                                  | <code>True</code>                         | Se enciende al inyectar la <code>}</code> de la raíz                                                        |

- Simplificación respecto a lo hablado:** no hacen falta `has_digit`, `dot_used` ni `quote_closed` como atributos. Los tres **se deducen de `_written`**: hay dígito si alguno de sus caracteres lo es · el punto se usó si está dentro · la comilla cerró si está dentro. Un atributo en vez de cuatro, y la simulación pasa a ser una sola string.
- Antes de empezar:** borra `_state` y `_stack` del archivo actual. `_stack` vuelve con el tipo de la tabla.

`__init__`

YA ESCRITO · SOLO COMPLETAR

```
def __init__(self, vocab: Dict[str, int], reversed_vocab: Dict[int, str],
              functions: List[Function]) -> None
```

RECIBE

El vocabulario en las dos direcciones y el catálogo ya validado por el `FileManager`.

- PASOS
- Guarda los tres argumentos. El catálogo se indexa por nombre — es lo único que ya está escrito.
  - Deja los cinco atributos de sesión en su **valor de reposo** (columna de la tabla). No los rellena: eso es de `start`.

**Por qué `_done` nace en `True`:** un `Guardian` recién construido, sin prompt, no tiene nada abierto. Así `is_open` devuelve `False` hasta que alguien llame a `start`, y ningún bucle arranca en falso.

start

UNA VEZ POR PROMPT

```
@validate_call
def start(self, prompt: str) -> None
```

RECIBE

El prompt del usuario, tal cual sale del `FileManager`.

PASOS

1. Pasa el prompt por `json.dumps`. Devuelve la string **ya entrecomillada y escapada**: se pega tal cual, sin añadirle comillas.
2. Monta `_json_str` con tres trozos: `{"prompt":` · el resultado del paso 1 · `, "name":` "
3. Vacía `_stack` y `_written`.
4. `_slot` pasa a "name" y `_done` se apaga.

#### DEJA

```
{"prompt": "What is the sum of 40 and 2?", "name": "
```

### is\_open

CONDICIÓN DEL WHILE DEL BLOQUE 5

```
def is_open(self) -> bool
```

#### PASOS

1. Devuelve lo contrario de `_done`.

**No se cuentan llaves de `_json_str`**. Un prompt que contenga `{` mete llaves dentro de una string y descuadra el saldo, y el prompt lo escribe el usuario. El único que sabe con certeza que el JSON cerró es quien inyecta la última llave: `_close_level`.

### \_closing\_char

PRIVADO

```
def _closing_char(self) -> str
```

#### RECIBE

Nada. Mira el último nivel de `_stack`.

#### PASOS

1. Coge el nodo y el índice del tope de la pila.
2. Si el índice **no** es el de la última clave del nodo → devuelve `,`
3. Si lo es → devuelve `}`

Este es todo el reparto: **Guardian decide cuál** de los dos caracteres se permite, y solo ese entra en la lista blanca. **El modelo decide cuándo** usarlo, y eso es lo que dice que el valor terminó.

### \_char\_ok

PRIVADO · EL NÚCLEO

```
def _char_ok(self, char: str, written: str) -> bool
```

#### RECIBE

`char` Un carácter suelto, el candidato.

`written` Lo que llevaría escrito el hueco justo antes de ese carácter. **No es `_written`**: es la copia que va creciendo durante la simulación de un token.

#### PASOS

Según `_slot`, una de estas tres tablas. Cualquier carácter que no aparezca en su tabla, no pasa.

SI `_SLOT` ES "NAME"

| CHAR           | PASA SI                                                                         |
|----------------|---------------------------------------------------------------------------------|
| "              | <code>written</code> es, tal cual, una clave de <code>_functions</code>         |
| cualquier otro | alguna clave de <code>_functions</code> empieza por <code>written + char</code> |

SI `_SLOT` ES "NUMBER"

| CHAR                             | PASA SI                                                  |
|----------------------------------|----------------------------------------------------------|
| un dígito                        | siempre                                                  |
| .                                | written tiene algún dígito <b>y</b> no tiene ya un punto |
| el de <code>_closing_char</code> | written tiene algún dígito                               |

SI `_SLOT` ES "STRING"

| CHAR                             | PASA SI                                                                                        |
|----------------------------------|------------------------------------------------------------------------------------------------|
| "                                | written no contiene ya una comilla                                                             |
| el de <code>_closing_char</code> | written <b>sí</b> contiene una comilla                                                         |
| cualquier otro                   | written no contiene comilla, <b>y</b> el carácter no es barra invertida ni carácter de control |

**La barra y los saltos de línea no se ofrecen nunca.** Romperían el JSON y el valor de un argumento no los necesita: lo que no puede escribirse, no entra en la lista. Así no hay escapado que manejar.

**Ojo al orden en "string":** el carácter de cierre puede coincidir con contenido legítimo (una `,` dentro del texto). Mira primero si la comilla ya está en `written`: si está, el hueco solo admite el cierre; si no está, la coma es contenido normal.

## `_token_ok`

PRIVADO

```
def _token_ok(self, text: str) -> bool
```

### RECIBE

El texto completo de un token del vocabulario.

### PASOS

1. Arranca una copia de `_written`.
2. Por cada carácter del texto, en orden:
  - Si el carácter anterior ya cerró el hueco → devuelve `False`: un token no puede escribir **nada** después del cierre.
  - Si `_char_ok` dice que no → devuelve `False`.
  - Si pasa, pégalo a la copia y sigue.
3. Si llegó al final → `True`.

```
_slot "string", written "hello", cierre permitido ",",
```

|                          |                                                                      |          |
|--------------------------|----------------------------------------------------------------------|----------|
| token <code>,</code>     | → <code>"</code> cierra el contenido, <code>,</code> cierra el hueco | VÁLIDO   |
| token <code>", "</code>  | → el espacio va después del cierre                                   | inválido |
| token <code>",\n"</code> | → el salto va después del cierre                                     | inválido |

```
_slot "number", written "40.5"
```

|                       |                        |          |
|-----------------------|------------------------|----------|
| token <code>.5</code> | → el punto ya se usó   | inválido |
| token <code>25</code> | → dos dígitos seguidos | VÁLIDO   |

**La copia importa.** Aquí se prueban ~151.000 tokens de los que se va a usar uno: si la simulación tocara `_written`, el primer token descartado ya habría corrompido el estado.

## `get_valid_ids`

PÚBLICO · LO LLAMA EL BLOQUE 5

```
def get_valid_ids(self) -> List[int]
```

### RECIBE

Nada. Trabaja sobre el estado.

### PASOS

1. Recorre `_vocab`, que da texto e id de cada token.
2. Quédate con el id de los que `_token_ok` apruebe.
3. Devuelve la lista.

#### DEVUELVE

`List[int]` — el Bloque 5 la usa directa como índice de `numpy` para dejar vivos solo esos logits.

**Nunca se llama fuera de un hueco.** Cuando `_slot` es `None`, `Guardian` está inyectando y no se piden logits.

**Es el punto caro del proyecto** — todo el vocabulario, y cada token carácter a carácter. Ahí entra el cache del bonus 4 cuando llegue: **encima** de este método, sin tocarlo por dentro.

### add\_token

PÚBLICO · LO LLAMA EL BLOQUE 5

```
@validate_call
def add_token(self, token_id: int) -> None
```

#### RECIBE

El id que ganó el `argmax`, ya como `int` de Python: **lo convierte el Bloque 5**, porque `numpy` devuelve `np.int64` y `Guardian` no conoce `numpy`.

#### PASOS

1. Traduce el id a texto con `_reversed_vocab`.
2. Pega ese texto a `_json_str` y a `_written`.
3. Si el último carácter **no** cierra el hueco → termina aquí. Le toca otra vez al modelo.
4. Si cierra, según cuál sea:
  - la comilla del hueco `"name"` → llama a `_close_name`
  - una `,` → sube el índice del tope de `_stack` y llama a `_open_key`
  - una `}` → llama a `_close_level`

**La coma del modelo no se inyecta otra vez.** Cuando el modelo escribe `,`, esa coma ya entró en `_json_str` en el paso 2: `_open_key` solo pone la clave. La coma que sí inyecta `Guardian` es otra — la de `_close_level`, al volver de un nivel anidado.

### \_close\_name

PRIVADO · SE CERRÓ EL HUECO DEL NOMBRE

```
def _close_name(self) -> None
```

#### PASOS

1. El nombre es `_written` sin la comilla final.
2. Busca esa función en `_functions`.
3. Inyecta `,` `"parameters": {`
4. **Si la función no tiene parámetros** → inyecta `}}`, enciende `_done`, deja `_slot` en `None` y termina. Al modelo no se le pide nada más.
5. Si los tiene → `push` a `_stack` con el nodo `parameters` de la función e índice `0`, y llama a `_open_key`.

### \_open\_key

PRIVADO · EL QUE SE LLAMA A SÍ MISMO

```
def _open_key(self) -> None
```

#### PASOS

1. Coge nodo e índice del tope de `_stack`. La clave es la que ocupa esa posición entre las claves del nodo.
2. Inyecta la clave con sus comillas y sus dos puntos: `"key":`
3. Mira el `TypeSpec` de esa clave:
  - **Tiene properties** → inyecta `{`, hace `push` con ese `properties` e índice `0`, y **se llama a sí mismo**. Ahí termina.

- **No tiene** → es hoja: si el tipo es `string`, inyecta la comilla de apertura. Pon `_slot` al tipo, vacía `_written`, y para: le toca al modelo.

**La recursión es la única parte que no es lineal**, y es donde vive el bonus 7. Un `properties` dentro de otro no necesita ningún caso especial: cada vuelta apila un nivel más.

## `_close_level`

PRIVADO · EL MODELO ESCRIBIÓ }

```
def _close_level(self) -> None
```

### PASOS

1. `pop` de `_stack`.
2. **Si la pila quedó vacía** → inyecta la `}` de la raíz, enciende `_done`, deja `_slot` en `None` y termina.
3. Si no: sube el índice del nivel que quedó arriba.
4. ¿Le quedan claves?
  - **Sí** → inyecta `,` y llama a `_open_key`.
  - **No** → inyecta `}` y **se llama a sí mismo**.

**El modelo escribe una sola llave de cierre**. Si al cerrar esa hoja se acaban también dos niveles de arriba, esas `}` las inyecta este método encadenándose, sin volver a pedir logits.

## Recorrido plano

FN\_ADD\_NUMBERS · A Y B NUMBER

```
start      {"prompt": "What is the sum of 40 and 2?", "name": "
            _slot = "name"

modelo     fn · _add · _numbers          (varios tokens)
modelo     "                             cierra el nombre
_close_name inyecta , "parameters": {    push (parameters, 0)
_open_key   inyecta "a":                  _slot = "number"

modelo     4 · 0                          _closing_char = "," (queda b)
modelo     ,                             cierra la hoja
add_token   índice → 1
_open_key   inyecta "b":                  _slot = "number"

modelo     2                             _closing_char = "}" (b es la última)
modelo     }                             cierra la hoja
_close_level pop → pila vacía → inyecta } _done = True

{"prompt": "What is the sum of 40 and 2?", "name": "fn_add_numbers",
 "parameters": {"a": 40, "b": 2}}
```

```
schema parameters: a → object con properties {x: number, y: number}
                  b → number

_close_name      inyecta , "parameters": {      push (parameters, 0)
_open_key        la clave "a" tiene properties
                  inyecta "a": {                push (properties_a, 0)
                  se llama a sí mismo
_open_key        inyecta "x":                    _slot = "number"

modelo           1          _closing_char = "," (queda y)
modelo           ,
_open_key        inyecta "y":                    _slot = "number"

modelo           2          _closing_char = "}" (y es la última del nivel)
modelo           }
_close_level     pop → queda (parameters, 0) → índice → 1 → queda "b"
                  inyecta ,
_open_key        inyecta "b":                    _slot = "number"

modelo           3          _closing_char = "}" (b es la última)
modelo           }
_close_level     pop → pila vacía → inyecta }      _done = True

..."parameters": {"a": {"x": 1, "y": 2}, "b": 3}}
```

Fuera de este bloque

PARA QUE NO LO BUSQUES AQUÍ

| ASUNTO                                                                   | DE QUIÉN ES                                                          |
|--------------------------------------------------------------------------|----------------------------------------------------------------------|
| Convertir <code>np.int64</code> a <code>int</code>                       | Bloque 5, antes de llamar a <code>add_token</code>                   |
| Poner los <code>-inf</code> y hacer el <code>argmax</code>               | Bloque 5                                                             |
| Cortar si se agota el presupuesto de tokens                              | Bloque 5 — es dueño del bucle. <code>Guardian</code> no cuenta pasos |
| <code>json.loads</code> y validación <code>pydantic</code> del resultado | Bloque 5                                                             |
| <code>encode</code> / <code>decode</code>                                | Bloque 1. Aquí se traduce con los dos diccionarios y nada más        |
| Cache de la lista blanca (bonus 4)                                       | Encima de <code>get_valid_ids</code> , sin tocarlo por dentro        |

Diseño vivo en `workflow/PROJECT.md` → *Bloque 4 — Validez de tokens*. Si algo aquí choca con ese archivo, manda el archivo.